

Linux_Wk9/S1 Lecture #20

Topics Covered

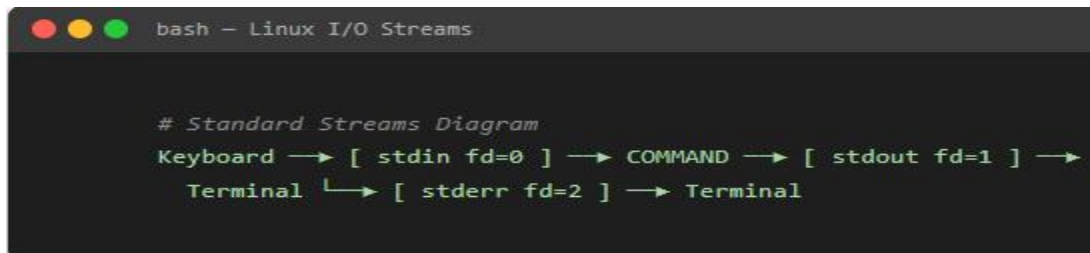
- Understanding Standard Streams (stdin, stdout, stderr)
 - Output Redirection (>) and Append (>>)
 - Input Redirection (<)
 - Error Redirection (2> and 2>>)
 - Combining stdout and stderr
 - The /dev/null Special File
 - Real-World Scenarios
 - Practice Exercises
 - Quick Reference Cheat Sheet.
-

➤ Understanding Standard Streams

Every Linux command has three built-in communication channels called standard streams. These are automatically connected when a process starts and form the foundation of all Linux I/O operation.

Stream	File Descriptor	Purpose	Default Source / Destination
stdin (Standard Input)	0	Data going INTO a command	Keyboard
stdout (Standard Output)	1	Normal output from a command	Terminal screen
stderr (Standard Error)	2	Error messages from a command	Terminal screen

- Visual Diagram



- Viewing stdout (normal output)

```
bash

user@linux:~$
echo "Hello World"
Hello World
```

Explanation: The echo command writes "Hello World" to stdout (file descriptor 1), which by default is connected to your terminal screen. You see the text appear on screen.

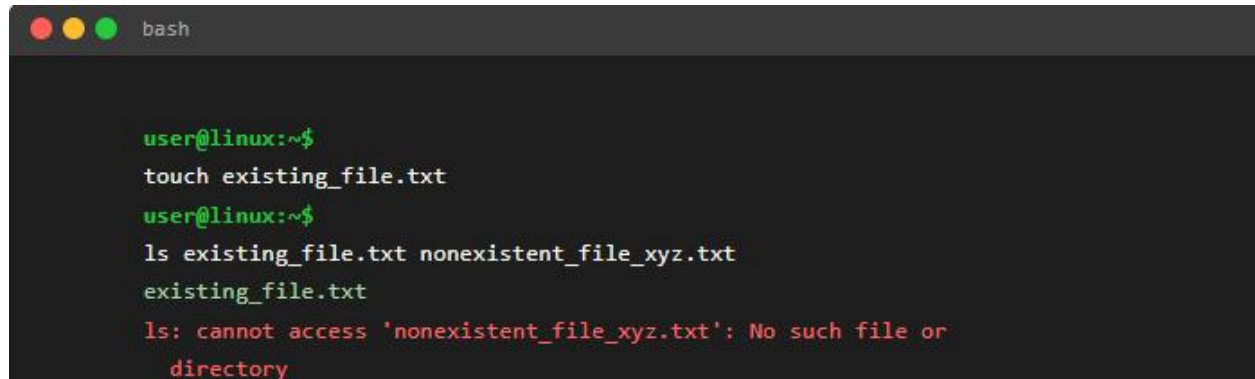
- Viewing stderr (error output)

```
bash

user@linux:~$
ls nonexistent_file_xyz.txt
ls: cannot access 'nonexistent_file_xyz.txt': No such file or
directory
```

Explanation: The ls command cannot find the file, so it writes an error message to stderr (file descriptor 2). By default, stderr also appears on your terminal screen.

- Viewing stderr (error output)



```
bash

user@linux:~$
touch existing_file.txt
user@linux:~$
ls existing_file.txt nonexistent_file_xyz.txt
existing_file.txt
ls: cannot access 'nonexistent_file_xyz.txt': No such file or
directory
```

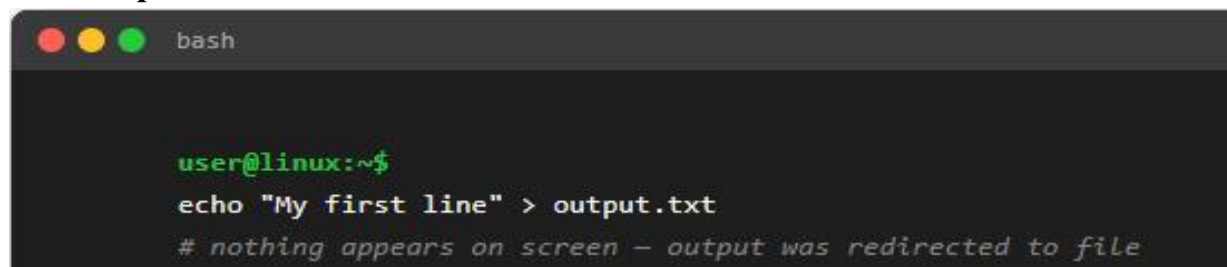
Explanation: The first line comes from stdout (successful result) and the second from stderr (error). Both appear on screen because both default to terminal, but they are separate streams that can be independently redirected.

- Output Redirection (>) and Append (>>)

- ◆ Basic Output Redirection (>)

The > operator redirects stdout (file descriptor 1) to a file. If the file already exists it is **overwritten** — content is destroyed. If it does not exist, it is created automatically.

- Simple redirection



```
bash

user@linux:~$
echo "My first line" > output.txt
# nothing appears on screen - output was redirected to file
```

Explanation: The > operator captured the stdout and sent it to output.txt instead of the screen.

- Verifying the file was created



```
bash

user@linux:~$
cat output.txt
My first line
```

Explanation: The cat command displays the contents of output.txt confirming "My first line" was successfully saved.

● Overwrite behaviour

```
bash

user@linux:~$
echo "Second line replaces first" > output.txt
user@linux:~$
cat output.txt
Second line replaces first
```

Explanation: The > operator always overwrites. The original content "My first line" is completely gone.

WARNING Using > on an existing file destroys its content instantly. There is no undo. Use >> if you want to preserve existing content.

● Redirecting ls output to a file

```
bash

user@linux:~$
ls /etc > etc_list.txt
user@linux:~$
head -5 etc_list.txt
acpi adduser.conf alternatives apache2 apm
```

Explanation: The ls /etc command lists all files in /etc. Instead of printing to screen, > saved the entire listing to etc_list.txt. We use head -5 to preview the first 5 lines.

● Redirect detailed listing

```
bash

user@linux:~$
ls -la /etc > etc_detailed.txt
user@linux:~$
head -5 etc_detailed.txt
total 1232 drwxr-xr-x 135 root root 12288 Mar 14 09:30 . drwxr-xr-x
20 root root 4096 Mar 10 14:22 .. drwxr-xr-x 3 root root 4096 Jan 15
08:20 acpi -rw-r--r-- 1 root root 3028 Jan 15 08:20 adduser.conf
```

Explanation: ls -la shows a detailed listing with permissions, sizes, and dates. We redirect to a file, then use head -5 to show only the first five lines.

◆ Append Operator (>>)

he >> operator redirects stdout to a file but **appends to the end** instead of overwriting. The original content is always preserved.

● Create initial file

```
bash

user@linux:~$ echo "Line 1" > append_test.txt
user@linux:~$ cat append_test.txt
Line 1
```

● Append second line

```
bash

user@linux:~$ echo "Line 2" >> append_test.txt user@linux:~$ cat append_test.txt
Line 1
Line 2
```

Explanation: The >> operator added "Line 2" to the end of the file. "Line 1" is still there.

● Build a log file progressively

```
bash

user@linux:~$ echo "Line 3" >> append_test.txt user@linux:~$ echo "Line 4" >>
append_test.txt user@linux:~$ echo "Line 5" >> append_test.txt user@linux:~$ cat
append_test.txt
Line 1
Line 2
Line 3
Line 4
Line 5
```

Explanation: Multiple >> operations build up the file content progressively. This is exactly how log files work — each event is appended so the full history is preserved.

TIP Use > when you want a fresh file each time (e.g., generating a report). Use >> when you want to accumulate data over time (e.g., log files, audit trails).

➤ Input Redirection (<)

The < operator makes a command read its input from a file instead of the keyboard. The file content is fed to the command's stdin (file descriptor 0).

◆ Basic Input Redirection

● Count lines via keyboard (no redirection)

```
bash
user@linux:~$ wc -l # wc waits for keyboard input... apple banana cherry # Press
Ctrl+D to signal end of input 3
```

Explanation: wc -l counts lines. Without <, it waits for you to type. You type 3 lines, press Ctrl+D (end-of-input signal), and it prints the count.

● Create an input file

```
bash
user@linux:~$ cat > fruits.txt << EOF apple banana cherry EOF # file created -
nothing printed to screen
```

Explanation: A here-document creates fruits.txt with three lines. EOF is just a delimiter marking where the input ends.

● Count lines from file using <

```
bash
user@linux:~$ wc -l < fruits.txt 3
```

Explanation: The < operator sends fruits.txt content as stdin to wc -l. Same result as typing manually, but fully automated. The file content becomes the input stream.

● Compare < with filename argument

```
bash
user@linux:~$ wc -l fruits.txt 3 fruits.txt # filename shown - wc opened file itself
user@linux:~$ wc -l < fruits.txt 3 # no filename - wc only sees data from stdin
```

Explanation: Without <, wc opens the file itself and also displays the filename. With <, the shell opens the file and feeds content to stdin — so wc never sees a filename, only data.

◆ Using Input Redirection with Different Commands

● Sort from file

```
bash
user@linux:~$ cat > unsorted.txt << EOF zebra apple mango banana EOF
user@linux:~$ sort < unsorted.txt apple banana mango zebra
```

Explanation: `sort < unsorted.txt` reads from the file via stdin, sorts alphabetically, and prints to stdout (screen).

● grep via input redirection

```
bash
user@linux:~$ cat > log.txt << EOF ERROR: disk full INFO: service started ERROR:
timeout INFO: backup done EOF user@linux:~$ grep 'ERROR' < log.txt ERROR: disk full
ERROR: timeout
```

Explanation: `grep` reads the file via stdin and prints only lines containing 'ERROR'. This is a practical pattern for filtering log files without pipes.

● tr via input redirection

```
bash
user@linux:~$ echo 'hello world' > lower.txt user@linux:~$ tr 'a-z' 'A-Z' < lower.txt
HELLO WORLD
```

Explanation: `tr` translates characters — here, converting lowercase to uppercase. It reads from the file via stdin and writes the transformed text to stdout.

NOTE Key difference summary:

`wc -l < file.txt` → Shell opens file, feeds to stdin. `wc` sees only data (no filename shown).

`wc -l file.txt` → `wc` opens the file itself. `wc` knows the filename and shows it in output.

➤ Error Redirection (2> and 2>>)

◆ Why Standard Redirection Doesn't Catch Errors

● Attempt to redirect error with >

```
bash
user@linux:~$ ls nonexistent_file.txt > error_capture.txt ls: cannot access
'nonexistent_file.txt': No such file or directory # stderr still leaks to screen -
only stdout was redirected user@linux:~$ cat error_capture.txt # file is empty - no
stdout was produced
```

Explanation: The > operator only redirects stdout (fd 1). The error message went to stderr (fd 2), which still points to the terminal. The file was created but is empty.

◆ Correct Error Redirection with 2>

● Redirect stderr to a log file

```
bash
user@linux:~$ ls nonexistent_file.txt 2> error.log # nothing on screen - error was
captured to file user@linux:~$ cat error.log ls: cannot access 'nonexistent_file.txt':
No such file or directory
```

Explanation: 2> specifically redirects file descriptor 2 (stderr). The error message is captured in error.log instead of appearing on screen.

◆ Separating stdout and stderr

● Create mixed scenario

```
bash
user@linux:~$ touch exists.txt user@linux:~$ ls exists.txt nonexistent.txt exists.txt
ls: cannot access 'nonexistent.txt': No such file or directory
```

Explanation: First line is stdout (success). Second line is stderr (error). Both appear on screen by default.

● Redirect stdout only (error still on screen)

```
bash
user@linux:~$ ls exists.txt nonexistent.txt > output_only.log ls: cannot access
'nonexistent.txt': No such file or directory # stderr still on screen
user@linux:~$ cat output_only.log exists.txt
```

Explanation: `>` captured only stdout ("exists.txt"). The error still appeared on screen because stderr wasn't redirected.

● Redirect stderr only (stdout still on screen)

```
bash
user@linux:~$ ls exists.txt nonexistent.txt 2> error_only.log exists.txt # stdout
still on screen user@linux:~$ cat error_only.log ls: cannot access 'nonexistent.txt':
No such file or directory
```

Explanation: `2>` captured only stderr (the error). The success output appeared on screen. Error is in file; success is on screen.

● Redirect both to separate files

```
bash
user@linux:~$ ls exists.txt nonexistent.txt > out.log 2> err.log # nothing on screen
- both streams redirected user@linux:~$ cat out.log exists.txt user@linux:~$ cat
err.log ls: cannot access 'nonexistent.txt': No such file or directory
```

Explanation: Complete separation — each stream in its own file. Nothing appears on screen.

◆ Appending Errors (2>>)

● Accumulate errors in a log

```
bash
user@linux:~$ ls file1.txt 2>> errors.log user@linux:~$ ls file2.txt 2>> errors.log
user@linux:~$ cat errors.log ls: cannot access 'file1.txt': No such file or directory
ls: cannot access 'file2.txt': No such file or directory
```

Explanation: Each `2>>` call appends the error to errors.log without overwriting previous entries. Ideal for persistent error logging across multiple command runs.

TIP In production scripts, redirect errors to a timestamped log file using:
command `2>> /var/log/myscript_$(date +%F).log` — this creates daily log files automatically.

➤ Combining stdout and stderr

◆ Redirect Both to the Same File

● Using 2>&1 (compatible with all shells)

```
bash
user@linux:~$ ls exists.txt nonexistent.txt > combined.log 2>&1 # nothing on screen - both streams in file
user@linux:~$ cat combined.log exists.txt ls: cannot access 'nonexistent.txt': No such file or directory
```

Explanation: `> combined.log` redirects stdout (1) to file. `2>&1` means "make stderr (2) go wherever stdout (1) currently goes" — which is the file. Both end up in `combined.log`.

● Using &> (bash 4+ shorthand)

```
bash
user@linux:~$ ls exists.txt nonexistent.txt &> combined2.log
user@linux:~$ cat combined2.log exists.txt ls: cannot access 'nonexistent.txt': No such file or directory
```

Explanation: `&>` is shorthand for redirecting both stdout and stderr to the same file. Cleaner syntax but less portable to older shells.

● Append both with &>>

```
bash
user@linux:~$ ls another_missing.txt &>> combined2.log
user@linux:~$ cat combined2.log exists.txt ls: cannot access 'nonexistent.txt': No such file or directory
ls: cannot access 'another_missing.txt': No such file or directory
```

Explanation: `&>>` appends both stdout and stderr to the file. Previous content is preserved and new output is added at the end.

◆ Critical: Order Matters with 2>&1!

WARNING The order of `2>&1` relative to `>` is critical. Wrong order = stderr goes to terminal instead of file.

● Wrong order (stderr escapes to terminal)

```
bash - WRONG ORDER EXAMPLE
```

```
user@linux:~$ ls exists.txt nonexistent.txt 2>&1 > wrong_order.log ls: cannot access  
'nonexistent.txt': No such file or directory # stderr leaked to screen! Only stdout  
went to file user@linux:~$ cat wrong_order.log exists.txt
```

Explanation: At `2>&1`, stdout still points to terminal so stderr is set to terminal. Then `>` redirects stdout to file but stderr is already locked to terminal.

● Correct order

```
bash - CORRECT ORDER EXAMPLE
```

```
user@linux:~$ ls exists.txt nonexistent.txt > right_order.log 2>&1 # nothing on  
screen - both streams in file user@linux:~$ cat right_order.log exists.txt ls: cannot  
access 'nonexistent.txt': No such file or directory
```

Explanation: `> right_order.log` redirects stdout to file first. Then `2>&1` redirects stderr to wherever stdout currently goes — the file. Both streams end up in the file — correct!

Syntax	Meaning	Portable?
<code>> file 2>&1</code>	Redirect stdout to file, then stderr to same file	Yes — all shells
<code>&> file</code>	Redirect both stdout and stderr to file (shorthand)	bash 4+ only
<code>>> file 2>&1</code>	Append stdout and stderr to file	Yes — all shells
<code>&>> file</code>	Append both stdout and stderr (shorthand)	bash 4+ only
<code>2>&1 > file</code>	WRONG ORDER — stderr goes to terminal!	N/A — avoid

➤ The /dev/null Special File

/dev/null is a special device file that discards all data written to it and returns EOF (end-of-file) when read. It is often called the "**bit bucket**" or "**black hole**" of Linux.

◆ What is /dev/null?

● Inspect the file

```
bash
user@linux:~$ file /dev/null /dev/null: character special (1/3) user@linux:~$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Mar 14 08:15 /dev/null
```

Explanation: It is a character special device (not a regular file). The c in permissions indicates a character device. All users can read and write to it.

◆ Discarding Output

● Discard stdout

```
bash
user@linux:~$ echo "This disappears" > /dev/null # nothing - output discarded into the void
```

Explanation: The output is written to /dev/null, which discards everything. Useful when you don't need to see the output of a command but still want it to run.

● Discard stderr

```
bash
user@linux:~$ ls nonexistent.txt 2> /dev/null # nothing - error message discarded
```

Explanation: The error message is sent to the void. Useful for clean scripts where you expect some errors and don't want them cluttering the output.

● Discard both stdout and stderr

bash

```
user@linux:~$ ls exists.txt nonexistent.txt &> /dev/null # nothing at all – both streams discarded
```

Explanation: Both success and error output are discarded. The command runs silently. Only the exit code (success/failure) remains meaningful.

◆ Emptying a File

● Truncate a file to zero bytes

bash

```
user@linux:~$ echo "Temporary data" > temp.log user@linux:~$ cat temp.log Temporary data user@linux:~$ > temp.log # file is now empty user@linux:~$ wc -c temp.log 0 temp.log
```

◆ **Explanation:** > file with nothing before it truncates the file to zero bytes instantly. This is faster than deleting and recreating the file, and preserves file permissions.

➤ Real-World Scenarios

The following scenarios demonstrate how I/O redirection is used in everyday Linux administration and scripting tasks.

◆ Scenario 1: Silent Existence Check

Check if a user exists without printing noisy output — show only your custom message.

```
bash - Scenario 1

# Check if user exists without showing command output user@linux:~$ id username &> /dev/null && echo 'User exists' || echo 'User not found' User not found
```

Explanation: `id` produces output and errors — we discard both with `&> /dev/null`, then use exit codes (`&&`, `||`) to print a clean custom message.

◆ Scenario 2: Silent Connectivity Test

Test internet connectivity without displaying ping output.

```
bash - Scenario 2

user@linux:~$ if ping -c 1 google.com &> /dev/null; then > echo 'Internet is up' > else > echo 'Internet is down' > fi Internet is up
```

Explanation: ping output is messy. We discard it with `&> /dev/null` and check only the exit code. Clean, professional output for users.

◆ Scenario 3: Rotating Log Files

Build a log rotation pattern that archives yesterday's log and starts fresh.

```
bash - Scenario 3

# Append timestamped entry to log user@linux:~$ echo "[$(date)] Service started" >> /var/log/myapp.log # Truncate log when it gets too large (over 1000 lines) user@linux:~$ [ $(wc -l < /var/log/myapp.log) -gt 1000 ] && > /var/log/myapp.log
```

Explanation: This pattern logs events with timestamps using `>>` and resets the log when it exceeds 1000 lines using `>` truncation.

◆ Scenario 4: Separating Script Audit Trails

Run a backup script that saves success output and errors to separate files.

```
bash - backup.sh

#!/bin/bash DATE=$(date +%F) rsync -av /data /backup > /var/log/backup_${DATE}.log \
2> /var/log/backup_${DATE}.err # Check if there were errors if [ -s
/var/log/backup_${DATE}.err ]; then echo "Backup had errors - check
backup_${DATE}.err" fi
```

Explanation: stdout (files transferred) goes to a dated log file. stderr (errors) goes to a separate .err file. The script checks if the error file has content and alerts accordingly.

◆ Scenario 5: Collecting System Information

```
bash - system_report.sh

REPORT='system_report.txt' > $REPORT echo "=== SYSTEM INFO ===" >> $REPORT uname -
a >> $REPORT echo "=== DISK USAGE ===" >> $REPORT df -h >> $REPORT echo "=== MEMORY
===" >> $REPORT free -h >> $REPORT echo "=== TOP PROCESSES ===" >> $REPORT ps aux --
sort=-%cpu | head -10 >> $REPORT 2>/dev/null user@linux:~$ cat $REPORT === SYSTEM
INFO === Linux hostname 5.15.0 #1 SMP x86_64 GNU/Linux === DISK USAGE === Filesystem
Size Used Avail Use% Mounted on /dev/sda1 50G 20G 28G 42% / === MEMORY === total used
free Mem: 3.7G 570M 3.1G
```

Explanation: This builds a complete system report by appending sections with >>. Errors are silently discarded. The final cat shows the assembled report.

◆ Scenario 6: Feeding Config as Input

```
bash - Scenario 6

# Count settings lines (excluding comments and blanks) user@linux:~$ grep -v '^#' <
/etc/ssh/sshd_config | grep -v '^$' | wc -l 42 # Sort a word list from file
user@linux:~$ sort -u < /usr/share/dict/words > sorted_unique_words.txt # file
created with sorted unique words
```

Explanation: grep -v removes comment lines and blank lines. The word list example reads from file via <, sorts and deduplicates, then saves to a new file with >.

➤ Practice Exercises

Complete these exercises to reinforce your understanding. Try to predict the output before running each command.

◆ Exercise Set A: Basic Redirection

● Exercise A1 — Beginner

Write commands to:

1. Create a file called my_info.txt containing the text "My Linux Journey"
2. Append the current date (use the date command) to the same file
3. Append the output of whoami to the file
4. Display the final file contents

bash – Exercise A1 Solution

```
user@linux:~$ echo "My Linux Journey" > my_info.txt user@linux:~$ date >> my_info.txt
user@linux:~$ whoami >> my_info.txt user@linux:~$ cat my_info.txt My Linux Journey
Sat Mar 14 10:30:00 UTC 2026 student
```

NOTE Expected my_info.txt contents: My Linux Journey / Sat Mar 14 10:30:00 UTC 2026 / student

● Exercise A2 — Beginner

Given this command: `ls /root /home /tmp` — predict what happens when you run each variation below, then verify:

1. Run: `ls /root /home /tmp > out.txt 2> err.txt` — which file gets what content?
2. Run: `ls /root /home /tmp > combined.txt 2>&1` — what is in combined.txt?
3. Run: `ls /root /home /tmp 2>&1 > combined2.txt` — how does this differ from above?

bash – Exercise A2 Solution

```
# Step 5: separate files user@linux:~$ ls /root /home /tmp > out.txt 2> err.txt
user@linux:~$ cat out.txt /home: student /tmp: user@linux:~$ cat err.txt ls: cannot
access '/root': Permission denied # Step 6: combined correct order user@linux:~$ ls
/root /home /tmp > combined.txt 2>&1 user@linux:~$ cat combined.txt ls: cannot access
'/root': Permission denied /home: student # Step 7: wrong order – stderr leaks to
screen user@linux:~$ ls /root /home /tmp 2>&1 > combined2.txt ls: cannot access
'/root': Permission denied # only stdout went to file, stderr leaked to terminal
```

◆ Exercise Set B: Intermediate

● Exercise B1 — Intermediate

Build a system health check script using only redirection (no pipes). The script should:

1. Clear or create a file called health_check.txt
2. Append hostname output with a section header
3. Append uptime output
4. Append disk usage for / only
5. Suppress any errors to /dev/null
6. Display the final report

```
bash - Exercise B1 Solution

user@linux:~$ > health_check.txt user@linux:~$ echo "=== HOSTNAME ===" >>
health_check.txt user@linux:~$ hostname >> health_check.txt 2>/dev/null
user@linux:~$ echo "=== UPTIME ===" >> health_check.txt user@linux:~$ uptime >>
health_check.txt 2>/dev/null user@linux:~$ echo "=== DISK USAGE ===" >>
health_check.txt user@linux:~$ df -h / >> health_check.txt 2>/dev/null
user@linux:~$ cat health_check.txt === HOSTNAME === linux-server === UPTIME ===
10:30:01 up 1:44, 2 users, load average: 0.00, 0.00, 0.00 === DISK USAGE ===
Filesystem Size Used Avail Use% Mounted on /dev/sda1 50G 20G 28G 42% /
```

● Exercise B2 — Intermediate

Create a word frequency counter using input redirection:

1. Create a file story.txt with at least 10 words, some repeated
2. Use tr with < to convert all text to lowercase, save to lower.txt
3. Use tr -s ' ' '\n' with < to put each word on its own line, save to words.txt
4. Use sort with < on words.txt to sort alphabetically, save to sorted_words.txt
5. Use wc -l with < to count total words

```
bash - Exercise B2 Solution

user@linux:~$ echo "the cat sat on the mat the cat" > story.txt user@linux:~$ tr 'A-
Z' 'a-z' < story.txt > lower.txt user@linux:~$ tr -s ' ' '\n' < lower.txt > words.txt
user@linux:~$ sort < words.txt > sorted_words.txt user@linux:~$ wc -l < words.txt 8
user@linux:~$ cat sorted_words.txt cat cat mat on sat the the the
```

◆ Exercise Set C: Advanced

● Exercise C1 — Advanced

bash – Exercise C1 Solution

```
user@linux:~$ ls /backup > backup_manifest.txt 2> backup_errors.log
user@linux:~$ echo "[$(date)] Manifest check" >> backup_manifest.txt
user@linux:~$ echo "[$(date)] Error check" >> backup_errors.log user@linux:~$ [ -s
backup_errors.log ] && echo "Errors found!" || echo "No errors." Errors found!
```

● Exercise C2 — Advanced (Challenge)

Explain what happens step-by-step when this command runs, then modify it so that errors are appended, results are sorted, and count is displayed on screen only:

bash – Exercise C2 Solution

```
# Original command user@linux:~$ find / -name '*.conf' > conf_files.txt 2>/dev/null #
Modified: errors appended (not overwritten) user@linux:~$ find / -name '*.conf' >
conf_files.txt 2>> find_errors.log # Sort results alphabetically user@linux:~$ sort <
conf_files.txt > sorted_conf.txt # Display count on screen only (not saved)
user@linux:~$ wc -l < conf_files.txt 147
```

TIP Hint for C2: You cannot sort the output and save it in one step without pipes. Instead, run find first, then sort the resulting file using `sort < conf_files.txt > sorted_conf.txt`

➤ Quick Reference Cheat Sheet

◆ Redirection Operators

Operator	Full Form	Action	Example
>	1>	Redirect stdout to file (overwrite)	echo hi > file.txt
>>	1>>	Redirect stdout to file (append)	echo hi >> file.txt
<	0<	Redirect file to stdin	sort < list.txt
2>	2>	Redirect stderr to file (overwrite)	cmd 2> err.log
2>>	2>>	Redirect stderr to file (append)	cmd 2>> err.log
2>&1	2>&1	Redirect stderr to same place as stdout	cmd > out 2>&1
&>	1> 2>	Redirect both stdout+stderr (overwrite)	cmd &> all.log
&>>	1>> 2>>	Redirect both stdout+stderr (append)	cmd &>> all.log

◆ File Descriptor Reference

FD	Name	Default	Redirect with
0	stdin	Keyboard	< file or 0< file
1	stdout	Terminal screen	> file or 1> file
2	stderr	Terminal screen	2> file

◆ Common Patterns

Goal	Command Pattern
Save output, show errors on screen	<code>command > output.log</code>
Show output on screen, save errors	<code>command 2> errors.log</code>
Save both to same file	<code>command > all.log 2>&1</code>
Save both to separate files	<code>command > out.log 2> err.log</code>
Suppress all output (silent run)	<code>command &> /dev/null</code>
Suppress errors only	<code>command 2> /dev/null</code>
Append output to existing log	<code>command >> app.log</code>
Append errors to existing log	<code>command 2>> errors.log</code>
Read input from file	<code>command < input.txt</code>
Truncate/empty a file	<code>> filename.txt</code>
Feed input + capture output	<code>command < in.txt > out.txt</code>
Capture all output, append mode	<code>command &>> combined.log</code>

◆ /dev/null Patterns

Pattern	Use Case
<code>cmd > /dev/null</code>	Discard stdout (keep errors visible)
<code>cmd 2> /dev/null</code>	Discard stderr (keep output visible)
<code>cmd &> /dev/null</code>	Discard everything (silent execution)
<code>cmd > /dev/null 2>&1</code>	Discard everything (portable form)
<code>> file.txt</code>	Empty/truncate a file to zero bytes

◆ Quick Diagnostic: Where Does Each Stream Go?

```
bash — Mental Model

# STEP 1: Where does stdout go?
Default: terminal | Changed by: > file or >> file

# STEP 2: Where does stderr go?
Default: terminal | Changed by: 2> file or 2>> file

# STEP 3: What is stdin?
Default: keyboard | Changed by: < file

# GOLDEN RULE: 2>&1 must come AFTER > file
Correct:  command > file 2>&1
Wrong:   command 2>&1 > file ← stderr leaks to terminal!
```

WrapUp & Key Takeaways

- Linux provides three standard streams — stdin (fd 0), stdout (fd 1), and stderr (fd 2) — that form the foundation of all I/O operations in the shell.
- The `>` operator redirects stdout to a file and **overwrites** existing content. Always use `>>` when you want to preserve existing data.
- The `<` operator feeds file content to a command's stdin. The command sees only data — not the filename — which differs from passing the filename as an argument.
- Error messages go to stderr (fd 2) by default. Use `2>` to capture errors separately from normal output.
- To redirect both stdout and stderr to the same file, use `> file 2>&1` (portable) or `&> file` (bash 4+ shorthand). Order matters — stdout must be redirected before `2>&1`.
- `/dev/null` is the "black hole" of Linux — anything written to it is silently discarded. It is essential for clean, professional scripts that suppress unwanted output.
- Real-world scripts combine these operators to separate audit trails, build system reports, rotate logs, and run silent background checks using only exit codes.
- Mastering I/O redirection improves shell scripting skills, troubleshooting ability, and system administration capabilities in all Linux environments.